

Chain-of-Thought & Reasoning Complexity

computational depth, in-context optimization, learning signals, reasoning-token lower bounds

이번 주의 질문은 “Chain-of-Thought(CoT)가 왜 잘 되는가?”를 하나의 메커니즘으로 설명하는 것이 아니다. 서로 다른 네 축을 분리한다. 첫째, autoregressive decoding이 추가하는 **순차 계산 깊이**. 둘째, Transformer의 한 forward pass 자체가 gradient descent나 backpropagation 같은 **학습 알고리즘을 내부에서 실행할 수 있는가**. 셋째, intermediate trace가 training에서 주는 **학습 신호와 sample-complexity** 이득. 넷째, inference에 필요한 **reasoning-token** 예산과 하한이다. 특히 이번 수정본에서는 두 번째 축 — ICL과 optimization/backprop 사이의 구성적 동치성 — 을 다른 세 축과 같은 비중으로 다룬다. 핵심은 “길게 생각하면 강해진다”가 아니라, **forward pass 내부의 optimization과 autoregressive reasoning**을 구분하고, 각각 무엇을 보장하는지를 정확히 나누는 것이다.

Primary readings

William Merrill and Ashish Sabharwal, **The Expressive Power of Transformers with Chain of Thought**, ICLR 2024.

Nirmit Joshi et al., **A Theory of Learning with Autoregressive Chain of Thought**, COLT 2025.

Awni Altabaa, Omar Montasser, and John Lafferty, **CoT Information: Improved Sample Complexity under Chain-of-Thought Supervision**, NeurIPS 2025.

Steve Hanneke, Idan Mehalel, and Shay Moran, **Sample Complexity of Autoregressive Reasoning: Chain-of-Thought vs. End-to-End**, 2026.

Kiran Tomlinson et al., **Reasoning about Reasoning: BAPO Bounds on Chain-of-Thought Token Complexity in LLMs**, ICML 2026.

Yu Bai et al., **Transformers as Statisticians: Provable In-Context Learning with In-Context Algorithm Selection**, 2023.

Weimin Wu et al., **In-Context Deep Learning via Transformer Models**, ICML 2025.

Contents

1	기호와 네 가지 이론 축	3
1.1	논문별 기호 대응표	3
1.2	CoT를 함수 반복으로 쓰기	3
1.3	네 가지 질문을 분리해야 한다	4
2	CoT는 순차 계산 자원이다	5
2.1	Week 5와의 연결: 한 번의 forward pass와 반복 decoding	5
2.2	DFA simulation이 보여주는 가장 단순한 이유	5
2.3	Turing-machine simulation과 계산 하한/상한	6
2.4	왜 “CoT = P”라고 단순화하면 안 되는가	6
3	ICL 안의 optimization loop: attention에서 backprop까지	7
3.1	출발점: linear attention 한 층은 GD 한 step을 구현할 수 있다	7
3.2	Bai et al.: GD를 base ICL algorithm으로 만들고 algorithm까지 선택한다	8
3.3	Wu et al.: forward pass 안에서 깊은 network의 backprop을 시뮬레이션한다	9
3.4	encoder나 decoder나보다 중요한 두 축	10
3.5	Chapter 2와 faithfulness에 어떻게 연결되는가	10
4	Autoregressive CoT를 학습한다는 것	12
4.1	Joshi et al.의 formal learning model	12
4.2	왜 E2E에서는 T 가 어려움을 키울 수 있는가	12
4.3	CoT supervision을 보면 무엇이 달라지는가	12
4.4	Littlestone dimension이 등장하는 이유	13
5	2026 결과: CoT sample complexity에서 T를 없애기	14
5.1	핵심 정리와 직관	14
5.2	E2E와 CoT를 섞지 말아야 한다	14
6	CoT Information: 중간 과정이 얼마나 informative한가	15
6.1	Hypothesis discrimination으로 보는 CoT	15
6.2	Sample complexity에서 어떻게 나타나는가	15
7	Reasoning token 자체의 하한: BAPO	16
7.1	왜 token lower bound가 필요한가	16
7.2	핵심 lower bound	16
7.3	학습 신호 축과의 대비	16
8	길게 생각하는 것의 비용과 오류 누적	18
8.1	필요한 reasoning length와 불필요한 verbosity는 다르다	18
8.2	Local error가 독립이라는 toy model	18
8.3	그래서 optimal reasoning budget 문제가 생긴다	18
8.4	Faithfulness 문제는 별도 축이다	19
9	세 이론을 하나의 그림으로 연결하기	20
9.1	Inference와 training을 한 표에 놓기	20

9.2	가장 중요한 다섯 개의 식	20
9.3	겉으로 모순처럼 보이는 문장 정리	20
9.4	발표에서 예상되는 질문	21
10	발표용 압축 정리	22
10.1	10분 설명 순서	22
10.2	칠판에 남길 식	22
10.3	마지막에 반드시 붙일 caveat	22
	Bibliography	23

1

기호와 네 가지 이론 축

1.1 논문별 기호 대응표

CoT 이론 논문은 같은 “길이”를 서로 다른 대상으로 쓴다. 이번 노트에서는 input length를 n , generation length를 T , 실제 reasoning-token budget을 R 로 구분한다.

개념	이 노트	Merrill-Sabharwal	Joshi / Hanneke / Altabaa
입력	$x \in \Sigma^*, x = n$	input string x	prompt x
기본 next-token rule	$f : \Sigma^* \rightarrow \Sigma$	fixed decoder block	base class $\mathcal{F} \subseteq \Sigma^{\Sigma^*}$
중간 추론	$z_{1:T}$	decoding steps / scratchpad	full CoT trajectory
최종 출력	y	last generated answer token	$f^{e2e,T}(x)$
CoT 전체	$f^{\text{CoT},T}(x)$	generated sequence	observed label under CoT supervision
복잡도 자원	T or R	decoding-step budget $t(n)$	sample size m , $\text{VCdim}(\mathcal{F})$
오차	ϵ	language decision error 보다 expressivity 중심	PAC target error / end-to-end risk

1.2 CoT를 함수 반복으로 쓰기

고정된 next-token generator

$$f : \Sigma^* \rightarrow \Sigma$$

가 있다고 하자. 입력 x 에서 autoregressive하게

$$z_1 = f(x), \quad z_2 = f(xz_1), \quad \dots, \quad z_T = f(xz_{1:T-1})$$

를 생성한다.

따라서 전체 chain과 최종 answer는

$$f^{\text{CoT},T}(x) = z_{1:T}, \quad f^{e2e,T}(x) = z_T$$

로 구분할 수 있다.

여기서 중요한 점은 모든 시점에 같은 f 가 쓰인다는 것이다. 이 time-invariance가 이후 sample complexity 결과의 핵심 가정이다.

1.3 네 가지 질문을 분리해야 한다

핵심 CoT 이론에서 서로 다른 네 질문을 섞지 않는다.

1. **Computational expressivity:** T 개의 decoding step을 허용하면 어떤 함수를 계산할 수 있는가?
2. **In-context optimization:** 한 번의 Transformer forward pass 안에 GD나 backprop 같은 학습 알고리즘을 얼마나 충실하게 구현할 수 있는가?
3. **Statistical learnability:** 그런 autoregressive rule을 학습하려면 training example이 몇 개 필요한가?
4. **Inference token complexity:** 이미 rule을 알고 있을 때 문제를 풀기 위해 reasoning token이 최소 몇 개 필요한가?

첫째는 계산복잡도, 둘째는 forward-pass 내부의 algorithmic simulation, 셋째는 sample complexity, 넷째는 inference-time lower bound다.

발표에서 이렇게 말하기 이번 주에는 CoT를 “추론 문장을 길게 쓰는 기법”이 아니라 자원 관점으로 봅니다. Decoding step은 serial compute를 추가하고, forward-pass depth는 context 안에서 optimization loop를 펼칠 수 있게 하며, intermediate supervision은 학습 신호를 추가하고, reasoning token budget은 inference 비용을 제한합니다. 이 네 효과는 수학적으로 서로 다른 문제입니다.

2

CoT는 순차 계산 자원이다

2.1 Week 5와의 연결: 한 번의 forward pass와 반복 decoding

Week 5에서 log-precision, fixed-depth Transformer의 한 번의 계산을 강하게 병렬화 가능한 회로로 보는 관점을 다뤘다. Week 7에서는 그 결과를 반복하지 않고, **decoding step**을 추가하면 무엇이 달라지는가에만 초점을 둔다.

Decoder가 한 step마다

$$y_{t+1} = F_{\theta}(x, y_{1:t})$$

를 수행하면, 모델 depth가 고정되어 있어도 $t = 1, 2, \dots, T$ 에 걸쳐 recurrent computation이 생긴다. 따라서 총 sequential depth는 대략

$$\text{effective sequential depth} \propto T$$

로 증가한다.

2.2 DFA simulation이 보여주는 가장 단순한 이유

Deterministic finite automaton을

$$A = (Q, \Sigma, \delta, q_0, F)$$

라고 하자. 문자열 $x_1 \cdots x_n$ 을 읽을 때 상태는

$$q_i = \delta(q_{i-1}, x_i)$$

로 갱신된다.

이 식은 본질적으로 recurrence다. 만약 reasoning token z_i 가 q_i 를 저장한다면

$$z_i = q_i$$

로 둘 수 있고, i 번째 decoding step에서

$$(q_{i-1}, x_i) \mapsto q_i$$

를 한 번 계산하면 된다.

Merrill–Sabharwal의 Theorem 1은 적절한 projected pre-norm, saturated attention 가정 아래 모든 regular language를 $|x| + 1$ decoding steps로 인식하는 decoder-only Transformer를 구성한다 [1].

원 논문 기준 이 결과는 “일반적인 상용 Transformer가 regular language를 항상 학습한다”는 뜻이 아니다. 존재성 construction이며, strict causal saturated attention과 projected pre-norm이라는 명시적 모델 가정이 있다. 핵심 이론 메시지는 **linear number of**

decoding steps가 recurrence를 가능하게 한다는 것이다.

2.3 Turing-machine simulation과 계산 하한/상한

Merrill-Sabharwal의 Theorem 2는 polynomially bounded time $t(n)$ 동안 동작하는 Turing machine M 에 대해, 한 decoding step으로 한 TM step을 simulate하는 Transformer construction을 준다. 따라서 논문 표현으로

$$\text{TIME}(t(n)) \subseteq \text{CoT}(t(n))$$

이다.

반대 방향에서는 계산을 simulate하는 비용을 세어

$$\text{CoT}(t(n)) \subseteq \widetilde{\text{TIME}}(n^2 + t(n)^2)$$

를 얻고, 공간 측면에서는

$$\text{CoT}(t(n)) \subseteq \text{SPACE}(t(n) + \log n)$$

를 얻는다 [1].

보충 유도 — 논문 밖의 독자용 전개 왜 space upper bound에 $t(n)$ 이 직접 들어가는지 직관적으로 확인해 보자. 생성한 intermediate token을 모두 buffer에 저장한다고 하면 필요한 저장량은

$$O(t(n)).$$

각 다음 token 계산을 위한 fixed-depth log-precision Transformer simulation에 logarithmic workspace가 필요하다고 두면

$$O(\log(n + t(n))).$$

따라서 총 workspace는

$$O(t(n) + \log(n + t(n))) = O(t(n) + \log n)$$

이다. 마지막 등식은 논문의 polynomially bounded $t(n)$ 가정 아래 이해하면 된다.

2.4 왜 “CoT = P”라고 단순화하면 안 되는가

Polynomial-length CoT가 polynomial-time computation까지 도달한다는 요약은 유용하지만, 반드시 다음 조건을 붙여야 한다.

- 모델은 논문에서 정의한 특정 decoder architecture다.
- precision, normalization, attention 방식에 대한 가정이 있다.
- 결과는 **representability**다. gradient-based training이 그 construction을 찾는다는 정리가 아니다.
- CoT length T 는 계산 자원이며, 실제 latency와 KV-cache cost는 별도다.

발표에서 이렇게 말하기 CoT가 주는 첫 번째 효과는 serial depth입니다. 한 번의 forward pass가 병렬 회로라면, autoregressive decoding은 그 회로를 반복 호출하면서 이전 출력 토큰을 다음 계산의 상태로 씁니다. 그래서 reasoning-token 수가 단순한 문장 길이가 아니라 계산 시간에 가까운 자원이 됩니다.

3

ICL 안의 optimization loop: attention에서 backprop까지

3.1 출발점: linear attention 한 층은 GD 한 step을 구현할 수 있다

먼저 “ICL이 학습한다”는 말을 가장 좁고 검증 가능한 형태로 쓴다. Reference predictor를

$$f_W(x) = Wx, \quad L(W) = \frac{1}{2N} \sum_{i=1}^N \|Wx_i - y_i\|^2$$

라고 하자. 그러면

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N (Wx_i - y_i)x_i^\top$$

이고, learning rate η 의 한 gradient-descent step은

$$\Delta W = -\eta \nabla_W L(W) = -\frac{\eta}{N} \sum_{i=1}^N (Wx_i - y_i)x_i^\top.$$

Query x_j 에서 prediction이 바뀌는 양은

$$\Delta W x_j = -\frac{\eta}{N} \sum_{i=1}^N (Wx_i - y_i)(x_i^\top x_j).$$

즉 각 context example의 residual과 query와의 similarity를 곱해 합한다.

von Oswald et al.의 핵심 observation은 linear self-attention에도 정확히 같은 bilinear sum이 있다는 것이다 [6]. 한 head를

$$e_j^+ = e_j + PVK^\top q_j = e_j + P \sum_{i=1}^N v_i(k_i^\top q_j)$$

로 쓰고 token을

$$e_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$

로 둔다. 다음 block projections을 택하자.

$$W_K = W_Q = \begin{bmatrix} I & 0 \\ 0 & 0 \end{bmatrix}, \quad W_V = \begin{bmatrix} 0 & 0 \\ W & -I \end{bmatrix}.$$

그러면

$$k_i^\top q_j = x_i^\top x_j, \quad v_i = \begin{bmatrix} 0 \\ Wx_i - y_i \end{bmatrix}.$$

Output projection의 y block을 $\eta I/N$ 로 잡으면 attention이 만드는 y -coordinate 변화는

$$\frac{\eta}{N} \sum_{i=1}^N (Wx_i - y_i)(x_i^\top x_j) = -\Delta W x_j.$$

따라서

$$e_j^+ = \begin{bmatrix} x_j \\ y_j - \Delta W x_j \end{bmatrix}$$

가 된다. 특히 query x_q 의 y -slot을 $-Wx_q$ 로 초기화하고 마지막에 부호를 뒤집어 readout하면

$$-(-Wx_q - \Delta W x_q) = (W + \Delta W)x_q$$

이므로 한 GD step 뒤의 prediction을 정확히 얻는다. 이 exact construction에서는 context token만 key/value에 넣는 설정(또는 $W \approx 0$ 인 단순화)도 함께 사용된다.

원 논문 기준 von Oswald et al. Proposition 1의 정확한 범위는 **one-head linear self-attention + 특정 regression tokenization**이다. Softmax를 제거한 attention에 대한 층의 data transformation과 MSE의 GD 한 step이 정확히 같아지도록 하는 parameter construction이 존재한다. 논문은 추가로 단순 regression setting에서 실제 meta-training이 이 construction에 가까운 해를 찾는 경우를 실험한다.

Akyürek et al.은 같은 문제를 더 일반적인 Transformer decoder primitive로 다룬다 [7]. Theorem 1은 한 in-context example에 대한 gradient update와 prediction을 constant number of layers, $O(d)$ hidden space로 구현할 수 있음을 construction으로 보인다. 여러 update는 이 block을 쌓아 구현한다. 따라서 초기 연구의 핵심 명제는

$$\text{context} \mapsto \text{task-specific optimizer state}$$

를 forward activations 안에 기록할 수 있다는 것이다.

3.2 Bai et al.: GD를 base ICL algorithm으로 만들고 algorithm까지 선택한다

Bai et al.은 “한 번의 GD step을 흉내 낸다”에서 두 단계 더 나아간다 [8]. 첫째, smooth convex empirical risk

$$\hat{L}_N(w) = \frac{1}{N} \sum_{i=1}^N \ell(w^\top x_i, y_i)$$

에 대해

$$w^{t+1} = w^t - \eta \nabla \hat{L}_N(w^t)$$

를 context 안에서 반복 실행하는 construction을 준다. Theorem 13의 핵심 형태는 L 번의 **GD iterate**를 $(L+1)$ -**layer attention-only Transformer**가 근사한다는 것이다. 한 attention layer가 한 in-context GD iterate를 구현하고, convexity/smoothness를 이용해 근사오차가 L 에 대해 선형으로 누적되도록 제어한다.

여기서 중요한 세부사항은 이론 construction이 표준 softmax가 아니라 **normalized ReLU attention**을 사용한다는 점이다. 즉

“Transformer가 GD를 한다”

보다

“특정 attention primitive가 GD iterate를 구현할 수 있다”

가 정확한 문장이다.

둘째, Bai et al.은 base algorithm 여러 개를 같은 Transformer가 context에 따라 선택하게 한다. 개념적으로 post-ICL validation은

$$\hat{k} = \arg \min_{k \in [K]} \hat{R}_{\text{val}}(A_k(D_{\text{train}}))$$

처럼 후보 ICL algorithm $A_1, \dots, A_K$ 를 실행한 뒤 validation signal로 고른다. 반대로 pre-ICL testing은 먼저 context statistic을 계산해

$$\hat{k} = g(s(D))$$

를 정하고 그 algorithm을 실행한다. 논문은 두 routing mechanism 모두에 explicit transformer construction을 준다.

핵심 이 결과가 주는 관점 변화는 “Transformer가 하나의 optimizer를 흉내 낸다”에서 “Transformer가 context 안에서 어떤 learning algorithm을 실행할지까지 선택할 수 있다”로의 확장이다. ICL을 단일 estimator가 아니라 algorithm-selection problem으로 볼 수 있다.

3.3 Wu et al.: forward pass 안에서 깊은 network의 backprop을 시뮬레이션한다

Linear regression의 한 step과 깊은 network의 backprop 사이에는 큰 간격이 있다. Wu et al.은 이 간격을 explicit construction으로 메운다 [9]. N -layer network의 parameter를 w 라 하고 empirical risk를 $L_n(w)$ 라 하자. Target inner optimizer는

$$w^{(\ell+1)} = \text{Proj}_{\mathcal{W}} \left(w^{(\ell)} - \eta \nabla L_n(w^{(\ell)}) \right).$$

한 GD step을 계산하려면 먼저 network의 forward propagation으로 모든 activation을 만들고, 그 뒤 chain rule을 거꾸로 적용해 각 layer의 gradient를 계산해야 한다. 논문의 construction은 이 계산을 Transformer depth에 펼친다.

1. 약 N 개 layer가 in-context example들의 forward activations를 계산한다.
2. 다음 약 N 개 layer가 backward error/chain-rule 항을 역순으로 전달한다.
3. 추가 layer들이 sample-wise gradient를 집계하고 parameter update와 projection을 수행한다.

Theorem 1의 정량적 statement는 다음 형태다.

$$L \text{ GD steps on an } N\text{-layer network} \iff (2N + 4)L\text{-layer Transformer}$$

그리고 각 simulated step은

$$\hat{w}^{(\ell)} = \text{Proj}_{\mathcal{W}} \left(\hat{w}^{(\ell-1)} - \eta \nabla L_n(\hat{w}^{(\ell-1)}) + \varepsilon^{(\ell-1)} \right), \quad \left\| \varepsilon^{(\ell-1)} \right\|_2 \leq \eta \epsilon$$

형태의 inexact GD update를 구현한다. Primary construction은 ReLU-attention 계열을 쓰며, 논문은 softmax Transformer로의 extension도 별도로 분석한다.

주의 arXiv:2411.16549의 현재 제목은 In-Context Deep Learning via Transformer Models이다. “How Do Transformers Learn In-Context Beyond Simple Functions?”는 arXiv:2310.10616의 별도 논문이다. $(2N + 4)L$ backprop simulation theorem은 arXiv:2411.16549의 Wu et al. 결과이므로 이 노트에서는 해당 원문을 기준으로 쓴다. 논문 본문은 “ReLU neural network”라는 표현을 쓰지만, Theorem 1의 formal assumptions에는 target activation r, r' 의 smoothness가 들어간다. 따라서 nonsmooth ReLU 자체에 대한 무조건적인 exact theorem으로 읽으면 안 된다.

3.4 encoder냐 decoder냐보다 중요한 두 축

이 계열의 논문을 architecture 이름만으로 분류하면 본질을 놓치기 쉽다.

논문	attention / architecture	내부에서 구현하는 것	핵심 한계
von Oswald et al.	linear self-attention	linear regression GD 1 step	exact construction이 softmax가 아님
Akyürek et al.	decoder construction	GD / ridge-related updates	linear-model setting 중심
Bai et al.	normalized ReLU attention; encoder 기본, decoder extension	multi-step GD + algorithm selection	주로 explicit existence construction
Wu et al.	ReLU construction + softmax extension	deep network의 forward/backprop + GD	depth가 $(2N + 4)L$ 로 증가

따라서 더 중요한 분류축은 다음 둘이다.

1. **linear / ReLU-style attention**에서 **softmax attention**으로 얼마나 가까이 가는가;
2. **얕은 estimator update**를 구현하는가, 아니면 깊은 **network**의 **forward/backward sweep** 전체를 구현하는가.

Encoder/decoder 차이는 causal mask와 information routing에 영향을 주지만, 이 theorem family의 중심 질문은 “forward computation 안에 어떤 optimizer circuit을 심을 수 있는가”이다.

3.5 Chapter 2와 faithfulness에 어떻게 연결되는가

Chapter 2의 CoT 결과는 여러 번의 **autoregressive forward pass**를 이어 붙여 serial computation을 만든다. 반면 지금 본 결과는 한 번의 **forward pass** 내부의 **layer depth**에 optimization loop를 펼친다. 두 자원을 구분해

$$D_{\text{inner}} = \text{optimizer/backprop을 구현하는 network depth},$$

$$T_{\text{decode}} = \text{autoregressive reasoning steps}.$$

라고 생각하는 편이 안전하다. 둘은 동시에 존재할 수도 있다.

또 “in-context으로 학습한다”는 말에도 세 수준이 있다.

1. **Representability**: 그런 optimizer와 같은 함수를 계산하는 Transformer parameter가 존재한다.
2. **Learnability of the circuit**: outer-loop SGD가 그 parameter/circuit을 실제로 찾는다.

3. **Mechanistic faithfulness:** 실제 자연 task에서 내부 computation이 그 optimizer라는 설명과 인과적으로 일치한다.

위 construction theorem들이 가장 강하게 보장하는 것은 1번이다. von Oswald와 Akyürek은 2번에 대한 실험적 evidence도 제공하지만, 1번에서 2번이나 3번이 자동으로 따라오지는 않는다. 이 점이 뒤의 faithfulness 결과 직접 연결된다.

발표에서 이렇게 말하기 ICL-backprop 동치성 측에서 중요한 건 encoder냐 decoder냐가 아닙니다. 한 쪽 끝에는 linear attention 한 층이 regression GD 한 step을 정확히 구현하는 결과가 있고, 다른 끝에는 $(2N + 4)L$ 층 Transformer가 N 층 network의 backprop을 L 번 시뮬레이션하는 결과가 있습니다. 즉 forward pass 자체가 작은 training loop를 품을 수 있다는 것이 핵심입니다. 다만 대부분은 “그런 circuit이 존재한다”는 정리이지, SGD가 실제 LLM에서 항상 그 circuit을 찾는다는 정리는 아닙니다.

4

Autoregressive CoT를 학습한다는 것

4.1 Joshi et al.의 formal learning model

Joshi et al.은 base next-token class

$$\mathcal{F} \subseteq \Sigma^{\Sigma^*}$$

를 두고, 한 $f \in \mathcal{F}$ 를 T 번 반복하여 얻는 end-to-end class를 연구한다 [2].

Training supervision은 두 가지다.

1. **E2E supervision:** (x, z_T) 만 관측한다.
2. **CoT supervision:** $(x, z_1, \dots, z_T)$ 전체를 관측한다.

같은 latent generator f_* 가 두 setting 모두의 ground truth다. 차이는 learner가 intermediate transition을 직접 보느냐이다.

4.2 왜 E2E에서는 T 가 어려움을 키울 수 있는가

T -step map을

$$\mathcal{F}^{\text{e2e}, T} = \{f^{\text{e2e}, T} : f \in \mathcal{F}\}$$

라고 하자. Base class가 단순하더라도 T 번 composition한 최종 mapping은 더 복잡해질 수 있다.

Joshi et al.은 binary alphabet에서 대략

$$\text{VCdim}(\mathcal{F}^{\text{e2e}, T}) \leq 6T \text{VCdim}(\mathcal{F})$$

형태의 bound를 보인다. 표준 realizable PAC bound를 쓰면

$$m_{\text{e2e}} = O\left(\frac{T \text{VCdim}(\mathcal{F}) \log(1/\epsilon) + \log(1/\delta)}{\epsilon}\right)$$

가 나온다.

원 논문 기준 더 중요한 것은 이 linear- T dependence가 단순 proof artifact 만은 아니라는 점이다. Joshi et al. Theorem 3.3은 주어진 VC dimension D 와 generation length T 에 대해, E2E learning이 $\Omega(DT)$ 규모의 sample dependence를 실제로 요구할 수 있는 base class를 구성한다.

4.3 CoT supervision을 보면 무엇이 달라지는가

한 trajectory

$$z = (x, z_1, \dots, z_T)$$

를 관측했다고 하자. 그러면 사실상 같은 base rule f_* 에 대한 transition constraint를 T 개 얻는다.

$$\begin{aligned}
f_{\star}(x) &= z_1, \\
f_{\star}(xz_1) &= z_2, \\
&\dots \\
f_{\star}(xz_{1:T-1}) &= z_T.
\end{aligned}$$

즉 하나의 trajectory는 final label 하나보다 훨씬 구조화된 정보를 준다. Joshi et al.의 CoT consistency rule은 training trajectory 전체와 일치하는 $\hat{f} \in \mathcal{F}$ 를 찾는다.

그 결과 binary case에서

$$m_{\text{CoT}} = O\left(\frac{\text{VCdim}(\mathcal{F}) \log T \log(1/\epsilon) + \log(1/\delta)}{\epsilon}\right)$$

형태의 bound를 얻는다. E2E의 최악 linear- T dependence가 $\log T$ 로 줄어든다.

4.4 Littlestone dimension이 등장하는 이유

E2E learning에서도 stronger combinatorial assumption을 두면 T dependence를 줄일 수 있다. Joshi et al.은

$$\text{VCdim}(\mathcal{F}^{\text{e2e}, T}) \lesssim \text{Ldim}(\mathcal{F}) \log T$$

형태의 결과를 보인다. 따라서

$$m_{\text{e2e}} = \tilde{O}\left(\frac{\text{Ldim}(\mathcal{F}) \log T}{\epsilon}\right)$$

가 가능하다.

왜 online-learning dimension이 나오는가? Autoregressive rollout에서는 learner가 이전 generated prefix에 따라 다음 query point를 스스로 만든다. 즉 fixed i.i.d. input classification보다 tree-structured sequential interaction에 가깝다.

주의 “CoT를 보면 sample complexity가 T 에 독립이다”는 문장은 Joshi et al. 2025 결과만 인용하면 정확하지 않다. 그 논문의 일반 VC-based CoT bound에는 $\log T$ 가 남는다. 완전한 T -independence는 이후 Hanneke–Mehalel–Moran 2026이 더 정교한 learning rule로 개선한다.

발표에서 이렇게 말하기 CoT supervision의 통계적 이점은 중간 문장이 친절해서가 아닙니다. 같은 time-invariant next-token rule에 대해 한 trajectory가 여러 transition constraint를 동시에 줍니다. 그래서 final answer만 보는 것보다 hypothesis를 훨씬 빨리 제거할 수 있습니다.

5

2026 결과: CoT sample complexity에서 T 를 없애기

5.1 핵심 정리와 직관

Hanneke–Mehalel–Moran은 Joshi et al.의 일반 VC-based CoT bound에 남아 있던 $\log T$ 가 필연적인지 묻는다 [4]. Finite-VC base class에서는 적절한 learner를 쓰면 **CoT supervision**의 **sample complexity**에서 **generation length T** 를 제거할 수 있다. 대표 형태는

$$m_{\mathcal{F}}^{\text{CoT}, T}(\epsilon, \delta) \leq c(\mathcal{F}) \left(\frac{1}{\epsilon} \log \frac{1}{\epsilon} + \log \frac{1}{\delta} \right),$$

여기서 $c(\mathcal{F})$ 는 base class의 combinatorial complexity에는 의존하지만 T 에는 의존하지 않는다. 직관은 한 trajectory

$$(x; z_1, \dots, z_T)$$

가 동일한 time-invariant rule f_* 에 대한 transition constraints

$$(x, z_1), (xz_1, z_2), \dots, (xz_{1:T-1}, z_T)$$

를 제공한다는 것이다. 이 transition들은 iid sample은 아니지만, compression 관점에서는 hypothesis를 결정하는 핵심 support set의 크기가 trajectory length와 함께 커질 필요가 없다.

5.2 E2E와 CoT를 섞지 말아야 한다

같은 결과가 end-to-end supervision에도 자동 적용되는 것은 아니다. E2E에서는 intermediate transition을 보지 못하므로 class 구조에 따라 T dependence가 constant에서 linear까지 다양하게 나타날 수 있다.

핵심 Inference에서 CoT를 생성하는 것은 **computational resource**를 추가하는 일이고, training에서 CoT trace를 label로 주는 것은 **statistical supervision**을 추가하는 일이다. Hanneke et al.의 T -independence는 두 번째 의미의 결과다. Token 수나 training FLOPs가 공짜라는 뜻은 아니다.

발표에서 이렇게 말하기 2025 결과에서는 CoT supervision bound에 $\log T$ 가 남았지만, 2026 후속 결과는 finite-VC base class에서 적절한 learner를 쓰면 sample 수의 T dependence 자체를 없앨 수 있음을 보입니다. 긴 trace가 더 많은 token 계산을 요구하는 것과, 더 많은 독립 training example을 요구하는 것은 다른 문제입니다.

6

CoT Information: 중간 과정이 얼마나 informative한가

6.1 Hypothesis discrimination으로 보는 CoT

Altabaa–Montasser–Lafferty는 CoT supervision의 이득을 “bad hypothesis를 얼마나 빨리 구분하는가”로 정량화한다 [3]. 두 hypothesis가 한 sample에서 CoT와 final answer 모두 같아 보일 확률을 p_{agree} 라 하면

$$\mathcal{I}_{\mathcal{D}}^{\text{CoT}}(h_1, h_2) = -\log p_{\text{agree}}.$$

Target h_* 에 대해 end-to-end error가 적어도 ϵ 인 후보만 모아

$$\mathcal{I}_{\mathcal{D}, h_*}^{\text{CoT}}(\epsilon; \mathcal{H}) = \inf_{\mathcal{R}^{\text{e2e}}(h) \geq \epsilon} \mathcal{I}_{\mathcal{D}}^{\text{CoT}}(h, h_*)$$

를 정의한다. 값이 클수록 틀린 hypothesis가 intermediate trace에서 빨리 드러난다.

6.2 Sample complexity에서 어떻게 나타나는가

Finite class에서 CoT-consistent hypothesis를 택하면 bad hypothesis 하나가 m 개 sample 동안 target과 구분되지 않을 확률은 대략

$$\exp\{-m\mathcal{I}_{\mathcal{D}}^{\text{CoT}}(h, h_*)\}$$

이다. Union bound로

$$m \gtrsim \frac{\log |\mathcal{H}| + \log(1/\delta)}{\mathcal{I}_{\mathcal{D}, h_*}^{\text{CoT}}(\epsilon; \mathcal{H})}$$

를 얻는다. 또한 agreement 사건은 “final answer도 같다”는 사건의 부분집합이므로

$$\mathcal{I}_{\mathcal{D}, h_*}^{\text{CoT}}(\epsilon; \mathcal{H}) \geq -\log(1 - \epsilon) \geq \epsilon.$$

따라서 realizable setting에서 informative한 CoT는 E2E signal보다 더 빠른 hypothesis elimination을 가능하게 할 수 있다.

주의 $\mathcal{I}^{\text{CoT}}$ 은 설명의 자연스러움, factuality, causal faithfulness를 재는 quantity가 아니다. 오직 statistical distinguishability를 측정한다. “중간 trace가 informative하다”와 “중간 trace가 내부 계산을 정직하게 설명한다”는 별개다.

발표에서 이렇게 말하기 CoT information은 reasoning 문장이 좋아 보이는지를 재는 점수가 아니라, 틀린 hypothesis를 training trace가 얼마나 빨리 배제하는지를 재는 통계적 양입니다. 그래서 sample complexity의 분모에 직접 들어갑니다.

7

Reasoning token 자체의 하한: BAPO

7.1 왜 token lower bound가 필요한가

앞의 sample-complexity 결과는 training example 수를 다뤘다. Tomlinson et al.은 BAPO, 즉 bounded attention prefix oracle이라는 모형을 사용해 이미 학습된 reasoning system의 **inference-time token complexity**를 묻는다 [5]. 한 step에서 prefix로 전달할 수 있는 summary와 직접 inspect할 수 있는 token 수를 제한하면, reasoning token 하나가 제한된 bandwidth의 communication round가 된다.

7.2 핵심 lower bound

Constant-bandwidth BAPO-CoT에서 논문은 대표적으로

$$\text{Majority} : \Omega(n), \quad \text{Match3} : \Omega(n),$$

그리고 graph reachability에서는 edge 수 m 에 대해

$$\text{Reachability} : \Omega(m)$$

형태의 lower bound를 보인다.

Proof의 공통 골격은 fooling pair다. 정답이 다른 두 input x, x' 를 만들되, R 이 너무 작으면 제한된 bandwidth 때문에 각 reasoning step에서 관찰 가능한 정보를 같게 유지한다. 그러면 induction으로

$$z_t(x) = z_t(x') \quad (t \leq R)$$

가 되어 final answer도 같아지는 contradiction이 생긴다. 따라서 충분한 communication rounds, 즉 reasoning tokens가 필요하다.

주의 BAPO theorem은 실제 GPT류 LLM 전체에 대한 lower bound가 아니다. 특정 bounded-information-flow abstraction에 대한 결과다. 또한 reachability의 정밀한 state-ment는 edge count m 을 쓰므로 “항상 $\Omega(n)$ ”으로 뭉뚱그리지 않는다.

7.3 학습 신호 축과의 대비

Hanneke 결과에서 CoT sample complexity가 T 에 독립일 수 있다는 사실과 BAPO의 $\Omega(n)$ token lower bound는 모순이 아니다.

$$\underbrace{m}_{\text{training examples}} \quad \text{와} \quad \underbrace{R}_{\text{inference reasoning tokens}}$$

는 다른 자원이다. 하나는 “규칙을 몇 trajectory로 배울 수 있는가”, 다른 하나는 “이미 배운 규칙으로 한 instance를 풀 때 몇 communication round가 필요한가”를 묻는다.

발표에서 이렇게 말하기 Reasoning token은 전부 불필요한 verbosity가 아닙니다. BAPO처럼 한 step의 information flow를 제한한 모델에서는 일부 문제에 선형 token 하한이 생깁니다. 따라서 training sample 수를 줄이는 것과 inference reasoning 길이를 줄이는 것은 전혀 다른 이론 문제입니다.

8

길게 생각하는 것의 비용과 오류 누적

8.1 필요한 reasoning length와 불필요한 verbosity는 다르다

앞 장의 lower bound는 어떤 task에는 긴 reasoning이 필요하다는 뜻이다. 그러나 필요 이상으로 token을 늘리는 것이 항상 이득이라는 뜻은 아니다.

총 inference cost를 거칠게

$$C_{\text{total}}(R) = C_{\text{prompt}} + \sum_{t=1}^R C_{\text{decode}}(t)$$

로 두자. KV cache를 사용해도 각 step의 attention/read cost는 context length와 무관하지 않다. 따라서 R 증가는 latency와 memory traffic을 늘린다.

8.2 Local error가 독립이라는 toy model

보충 유도 — 논문 밖의 독자용 전개 다음은 특정 논문의 정리가 아니라 오류 누적을 설명하기 위한 독자용 toy model이다. 각 reasoning step이 올바른 확률을 $1 - p$ 라 하고, 모든 step이 맞아야 final answer가 맞으며 step error가 독립이라고 가정하자. 그러면 chain 전체 성공 확률은

$$P_{\text{succ}}(R) = (1 - p)^R.$$

작은 p 에 대해

$$\log P_{\text{succ}}(R) = R \log(1 - p) \approx -pR,$$

따라서

$$P_{\text{succ}}(R) \approx e^{-pR}.$$

즉 local reliability가 고정되어 있으면 chain이 길수록 누적 실패 확률은 커질 수 있다. 실제 reasoning model은 error correction, backtracking, self-verification 때문에 이 독립 모형과 다르다.

8.3 그래서 optimal reasoning budget 문제가 생긴다

Task 난이도에 따라 필요한 최소 계산량을 $R_{\min}(x)$ 라고 하자. 그보다 짧으면 computational lower bound 때문에 실패할 수 있고, 너무 길면 비용과 오류 누적이 커질 수 있다.

개념적으로는

$$R^*(x) = \arg \min_R [\mathcal{R}_{\text{task}}(R \mid x) + \lambda C_{\text{total}}(R)]$$

같은 문제로 볼 수 있다.

이 식은 이번 주 primary paper의 theorem이 아니라 inference-time compute allocation을 연결하기 위한 모델링 식이다. 그럼에도 중요한 메시지는 분명하다.

핵심 “더 긴 CoT”와 “더 많은 유효 계산”은 동일하지 않다. 필요한 serial depth를 확보하는 token과, 이미 충분한 계산 이후의 verbosity를 구분해야 한다. 이 distinction이 adaptive reasoning / early stopping / verifier 연구로 연결된다.

8.4 Faithfulness 문제는 별도 축이다

Visible CoT가 실제 내부 computation을 정확히 반영하는가도 별개의 질문이다. 이번 주의 complexity theorem들은 intermediate token을 computational state 또는 supervision signal로 취급하고, 앞의 ICL-optimization construction은 특정 forward function이 GD/backprop과 같아질 수 있음을 보였다. 둘 모두에서 **functional equivalence**와 **mechanistic explanation**을 구분해야 한다.

함수적 동치만으로 내부 메커니즘까지 결론내릴 수는 없다.

GD/backprop과 같은 output을 계산한다 ≠ 그 알고리즘이 실제 learned circuit이다,
CoT가 계산에 유용하다 ≠ visible CoT가 내부 인과 과정을 설명한다.

따라서 “ICL이 진짜 학습인가?”라는 질문에는 먼저 의미를 고정해야 한다. Context에 따라 output이 optimizer update와 함수적으로 동치라는 의미라면 construction theorem이 답할 수 있다. 반면 outer-loop SGD가 그 circuit을 찾았는지, visible rationale가 그 hidden computation을 설명하는지는 별도의 mechanistic/causal question이다.

발표에서 이렇게 말하기 Reasoning length에는 양면성이 있습니다. 너무 짧으면 필요한 serial computation을 못 하고, 너무 길면 latency와 오류 누적이 늘 수 있습니다. 그래서 중요한 질문은 “CoT가 긴가 짧은가”가 아니라, task마다 필요한 최소 계산량과 실제 사용한 계산량의 gap입니다.

9

세 이론을 하나의 그림으로 연결하기

9.1 Inference와 training을 한 표에 놓기

관점	질문	핵심 자원	대표 결과
Computational expressivity ICL $\leftrightarrow$ optimization	CoT를 허용하면 무엇을 계산 가능한가? 한 forward pass가 learning algorithm을 실행할 수 있는가?	decoding steps T layer depth, attention primitive	$\text{TIME}(t) \subseteq \text{CoT}(t)$ under assumptions 1-layer GD부터 $(2N + 4)L$ backprop simulation까지
Autoregressive learnability	T -step rule을 몇 sample로 학습하는가?	examples m , $\text{VCdim}(\mathcal{F})$	CoT supervision은 T dependence를 줄이거나 제거 가능 sample complexity $\sim d/\mathcal{I}^{\text{CoT}}(\epsilon)$
CoT information	trace가 bad hypothesis를 얼마나 잘 구별하는가?	$\mathcal{I}^{\text{CoT}}(\epsilon)$	
Inference token lower bound	최소 몇 reasoning token이 필요한가?	token budget R	BAPO-hard tasks에서 linear lower bounds

9.2 가장 중요한 다섯 개의 식

Autoregressive recurrence:

$$z_{t+1} = f(xz_{1:t}).$$

Serial computation:

$$\text{TIME}(t(n)) \subseteq \text{CoT}(t(n)).$$

In-context gradient step:

$$\Delta W = -\frac{\eta}{N} \sum_i (Wx_i - y_i)x_i^\top.$$

Linear attention은 prediction에 같은 update를 구현할 수 있다.

Deep backprop simulation:

$$\begin{aligned} &L \text{ GD steps on an } N\text{-layer network} \\ &\iff (2N + 4)L \text{ Transformer layers.} \end{aligned}$$

Learning/inference complexity:

$$m_{\text{CoT}} \text{ can be independent of } T, \quad R_{\text{reason}} = \Omega(n) \text{ on BAPO-hard tasks.}$$

9.3 걸으로 모순처럼 보이는 문장 정리

1. “긴 CoT는 계산 능력을 늘린다.” — 가능. Expressivity 관점이다.

2. “긴 CoT는 더 많은 training sample을 요구한다.” — **필수 아님**. CoT supervision에서는 T -independent bound가 가능하다.
3. “CoT는 sample complexity를 줄인다.” — **조건부**. Intermediate trace가 informative해야 한다.
4. “reasoning token은 압축하면 된다.” — **항상 아님**. 일부 task에는 token lower bound가 있다.
5. “Transformer가 GD/backprop을 구현할 수 있으니 실제 SGD가 그 circuit을 찾는다.” — **따라오지 않음**. Existence와 optimization은 별도다.
6. “CoT가 유용하니 설명도 faithful하다.” — **따라오지 않음**. Faithfulness는 별도 성질이다.

9.4 발표에서 예상되는 질문

Q1. Week 5의 computational theory와 이번 주가 겹치지 않나?

Week 5의 목적은 Transformer 계산 모델의 upper/lower class를 이해하는 것이었다. 이번 주는 CoT를 중심으로 **계산 자원 + forward-pass 내부 optimization + 학습 복잡도 + token lower bound**를 연결한다. 특히 ICL-GD/backprop construction과 Joshi/Hanneke/Altabaa/BAPO가 새 연결점이다.

Q2. CoT sample complexity가 T 에 독립이면 긴 trace가 공짜인가?

아니다. Training example 수가 T 에 독립일 수 있다는 뜻이지, 한 sample 안의 token 수나 training FLOPs가 독립이라는 뜻이 아니다.

Q3. BAPO lower bound가 실제 GPT류 LLM에 그대로 적용되나?

아니다. BAPO는 bounded information flow를 추상화한 모델이다. 실제 model experiment와 정성적 일치는 보고되지만 theorem의 직접 적용이라고 말하면 과장이다.

Q4. ICL-GD 동치성이 실제 LLM도 backprop을 한다는 뜻인가?

아니다. von Oswald/Bai/Wu의 핵심 정리는 특정 architecture에 대한 explicit construction이다. 일부 단순 regression 실험에서는 학습된 weight와 construction의 정렬 evidence가 있지만, 자연어 LLM 전체에 대한 mechanistic identity는 아니다.

Q5. CoT information이 크면 무조건 좋은 reasoning trace인가?

통계적 hypothesis discrimination 측면에서는 유리하다. 하지만 human interpretability, factuality, causal faithfulness를 직접 측정하는 양은 아니다.

발표에서 이렇게 말하기 이번 주 전체를 한 문장으로 정리하면 이렇습니다. Transformer는 한 forward pass 안에 optimizer/backprop circuit을 구현할 수 있고, autoregressive CoT는 그 forward pass를 반복해 serial compute를 추가합니다. Training trace는 hypothesis를 더 빨리 식별하게 만들 수 있지만, inference reasoning token에는 별도의 task-dependent lower bound가 있습니다.

10

발표용 압축 정리

10.1 10분 설명 순서

1. **1분**: CoT를 $z_{t+1} = f(x, z_{1:t})$ 라는 repeated computation으로 정의.
2. **2분**: Merrill-Sabharwal — decoding step이 serial computation resource라는 점.
3. **3분**: ICL $\leftrightarrow$ optimization — linear attention의 1-step GD, Bai의 algorithm selection, Wu의 $(2N + 4)L$ backprop simulation.
4. **2분**: Joshi/Hanneke/Altabaa — intermediate trace가 sample complexity와 hypothesis discrimination을 어떻게 바꾸는가.
5. **2분**: BAPO + caveat — inference token lower bound, existence vs learned mechanism, faithfulness 구분.

10.2 칠판에 남길 식

$$z_{t+1} = f(xz_{1:t}), \quad \text{TIME}(t(n)) \subseteq \text{CoT}(t(n)).$$

$$\Delta W = -\frac{\eta}{N} \sum_i (Wx_i - y_i)x_i^\top \iff \text{one linear-attention GD update.}$$

L GD steps on an N -layer network

$\Leftarrow (2N + 4)L$ Transformer layers.

$$m^{\text{CoT}, T} \text{ can be independent of } T, \quad m \sim \frac{d}{\mathcal{I}^{\text{CoT}}(\epsilon)},$$

$$R_{\text{reason}} = \Omega(n) \quad \text{on BAPO-hard tasks.}$$

10.3 마지막에 반드시 붙일 caveat

주의 이번 주의 theorem들은 서로 다른 formal model을 사용한다. Merrill-Sabharwal의 decoder construction, von Oswald/Bai/Wu의 in-context optimizer constructions, Joshi/Hanneke의 PAC model, Altabaa의 CoT hypothesis class, Tomlinson et al.의 BAPO abstraction을 하나의 universal theorem처럼 합치면 안 된다. 특히 **optimizer**를 구현하는 **weight**가 존재한다와 **SGD**가 실제로 그 **weight**를 찾는다는 구분을 해야 한다.

발표에서 이렇게 말하기 CoT 이론에서 중요한 구분은 네 가지입니다. 몇 decoding step을 쓰면 무엇을 계산할 수 있는가, 한 forward pass 안에 어떤 optimizer를 구현할 수 있는가, 그 rule을 몇 sample로 학습할 수 있는가, 그리고 실제 문제를 풀 때 최소 몇 reasoning token이 필요한가입니다. 이 네 질문을 분리하면 ICL과 CoT 결과들이 한 그림 안에서 충돌 없이 정리됩니다.

Bibliography

- [1] William Merrill and Ashish Sabharwal. The Expressive Power of Transformers with Chain of Thought. International Conference on Learning Representations (ICLR), 2024.
- [2] Nirmal Joshi, Gal Vardi, Adam Block, Surbhi Goel, Zhiyuan Li, Theodor Misiakiewicz, and Nathan Srebro. A Theory of Learning with Autoregressive Chain of Thought. Proceedings of the 38th Conference on Learning Theory (COLT), PMLR 291:3161–3212, 2025.
- [3] Awni Altabaa, Omar Montasser, and John D. Lafferty. CoT Information: Improved Sample Complexity under Chain-of-Thought Supervision. Advances in Neural Information Processing Systems (NeurIPS), 2025.
- [4] Steve Hanneke, Idan Mehal, and Shay Moran. Sample Complexity of Autoregressive Reasoning: Chain-of-Thought vs. End-to-End. arXiv:2604.12013, 2026.
- [5] Kiran Tomlinson, Tobias Schnabel, Adith Swaminathan, and Jennifer Neville. Reasoning about Reasoning: BAPO Bounds on Chain-of-Thought Token Complexity in LLMs. International Conference on Machine Learning (ICML), 2026.
- [6] Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers Learn In-Context by Gradient Descent. Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR 202, 2023. arXiv:2212.07677.
- [7] Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What Learning Algorithm Is In-Context Learning? Investigations with Linear Models. International Conference on Learning Representations (ICLR), 2023. arXiv:2211.15661.
- [8] Yu Bai, Fan Chen, Huan Wang, Caiming Xiong, and Song Mei. Transformers as Statisticians: Provable In-Context Learning with In-Context Algorithm Selection. arXiv:2306.04637, 2023.
- [9] Weimin Wu, Maojiang Su, Jerry Yao-Chieh Hu, Zhao Song, and Han Liu. In-Context Deep Learning via Transformer Models. Proceedings of the 42nd International Conference on Machine Learning (ICML), PMLR 267, 2025. arXiv:2411.16549.
- [10] William Merrill and Ashish Sabharwal. The Parallelism Tradeoff: Limitations of Log-Precision Transformers. Transactions of the Association for Computational Linguistics, 2023.